ProxiSync

Build Guide: BLE-Based Anti-Proxy Attendance System

Complete plan, roadmap, backend contract and frontend specification

Project type	Minor project, B.Tech 2nd year, CSE (AIML)
Stack	React Native (Android first), Node.js + Express, PostgreSQL
Core idea	Teacher phone broadcasts a rotating BLE token. Student phone scans it, signs the request with a device-bound key, and the server runs a multi-layer check before saving attendance.
Audience	Whole team. Section 7 onward is written for the frontend developer.

1. Summary of the plan so far

This section recaps everything decided in the planning stage so every team member starts from the same page.

1.1 The problem and the idea

- Manual attendance wastes class time, allows proxy attendance (a friend marks you present) and produces error-prone records.
- ProxiSync proves two things: the student is physically in the room (BLE proximity) and the student is who they claim to be (device-bound key).
- The teacher app broadcasts a short-lived token over Bluetooth Low Energy. The student app scans it and sends it to the server with a signature. The server decides.

1.2 Three problems found in the original deck, and the fixes

Problem	Why it fails	Fix adopted
Phones as BLE beacons	Android can advertise, but iOS restricts custom advertising, and React Native peripheral support is weak.	Android-only for the demo. Test advertising in Week 1. Fallback: one cheap ESP32 beacon per classroom.
Binding to hardware MAC address	Modern phones randomize MAC addresses and apps cannot read the real one.	On registration the phone generates a key pair in its secure keystore. The server stores only the public key. Every attendance request is signed with the private key.
Static session token	A token can be forwarded over WhatsApp to a friend outside the room.	Token rotates every 20 seconds. Server accepts the current and previous slot only.
RSSI used alone	Walls, bodies and phone models shift the reading by 10 dB or more.	RSSI is a coarse filter: median of at least 3 readings against a per-room threshold calibrated in Week 1.

1.3 The verification shield (server side, in this order)

#	Check	Rejection reason code
1	Session exists, is not closed, and is inside its time window	SESSION_INACTIVE
2	Student is enrolled in the session's course	NOT_ENROLLED
3	Request timestamp is within 30 seconds of server time (anti-replay)	STALE_REQUEST
4	Student has a bound device, and the signature matches its public key	NO_DEVICE_BOUND, BAD_SIGNATURE
5	Rotating token matches the current or previous 20-second slot	TOKEN_EXPIRED_OR_INVALID
6	At least 3 RSSI samples and their median is above the room threshold	TOO_FAR
7	Student has not already been marked for this session	ALREADY_MARKED

1.4 Current project status

Part	Status
Backend API, database schema, attack-test simulator	Written. Not yet run against a database. Only syntax-checked.
Student app and teacher app (frontend)	Not built. This is the next major work item.
BLE advertise and scan on real phones	Not built or tested.

Part	Status
Classroom RSSI measurements	Not done.
Report, slides, demo video	Not done.

2. Environment setup (Windows)

2.1 Install the tools

- **Node.js LTS** from nodejs.org. Close and reopen Command Prompt afterwards. Check with `node -v` and `npm -v`.
- **PostgreSQL** from postgresql.org/download/windows. Remember the password set for the postgres user. It also installs pgAdmin.
- **Android Studio** (for the Android SDK, emulator and USB debugging drivers) and **JDK 17**, needed for React Native.
- **Git** and VS Code.
- A **real Android phone** (Android 8 or newer) per team member for testing. BLE does not work on emulators.

2.2 Create the database

- Open pgAdmin, log in, right-click Databases, Create, Database, name it `proxisync`.
- Right-click the new database, open Query Tool, paste the contents of `schema.sql`, and run it (F5).

2.3 Start the backend

```
cd proxisync-backend
npm install
set DATABASE_URL=postgres://postgres:YOUR_PASSWORD@localhost:5432/proxisync
set JWT_SECRET=anything123
npm start
```

You should see *ProxiSync API running*. In a second Command Prompt run `npm run simulate`. It prints an INSERT statement that enrolls the test students: run it in pgAdmin, then press Enter. All five test lines should print PASS.

For phone testing the phone and laptop must be on the same WiFi. Use the laptop's local IP address (run `ipconfig`, look for IPv4), for example `http://192.168.1.10:3000`. Allow port 3000 in Windows Firewall if the phone cannot connect.

3. Step-by-step: what to build

Work in this order. Each step has a clear finish line so the team can tell when it is done. Nothing in later steps should start before Step 1 passes.

Step 1 - Feasibility spike (Week 1)

Goal: Prove the core idea works on real phones.

- Build a bare React Native test app (no UI polish).
- Make one phone advertise a BLE packet containing a test payload. Make a second phone scan and print the payload and RSSI.
- Record RSSI at 1 m, 3 m, 6 m and 10 m inside the real classroom, with several readings each.
- Decide: phone-as-beacon or ESP32 beacon. Decide Android-only or Android plus iOS.

Done when: A table of RSSI against distance, and a written decision on the beacon approach.

Step 2 - Backend running (Week 2)

Goal: Get the existing backend working end to end.

- Complete Section 2 and run the simulator.
- Fix any errors that appear on first run.
- Add the extra endpoints listed in Section 6.3 (courses, rooms, students, history) so the apps have data to show.
- Create a Postman collection for every endpoint.

Done when: All five simulator tests print PASS and every endpoint works from Postman.

Step 3 - Student app (Weeks 3-4)

Goal: Login, device registration, scan and submit.

- Project setup, navigation and API client (Section 8).
- Login and register screens.
- First-login device registration: create key pair, send public key.
- Mark Attendance screen: scan, collect RSSI samples, sign, submit.
- My Attendance screen with percentage per subject.

Done when: A student can mark attendance end to end on a real phone against the real server.

Step 4 - Teacher app (Weeks 4-5)

Goal: Session control and live view.

- Login and course selection.
- Start Session: create session, advertise the rotating token.
- Live Session screen: countdown, live list of present students, Close button.
- Manual add with a required note.
- Reports screen (CSV).

Done when: A full class: teacher starts, five or more students mark, teacher closes and sees the list.

Step 5 - Security and calibration (Week 6)

Goal: Prove the system resists cheating.

- Set each room's RSSI threshold from the Step 1 measurements.
- Run the attack tests on real phones: forwarded token, friend's phone, replayed request, student in the corridor.
- Test edge cases: Bluetooth off, permissions denied, no internet, teacher phone dies.
- Device-change flow: teacher resets a student's device, student re-registers.

Done when: Every attack is rejected with the expected reason code, and edge cases show a clear message.

Step 6 - Field test, report and demo (Week 7)

Goal: Evidence for the review.

- Run a real class with 15-20 students. Record false accepts, false rejects and time to mark.
- Write the report: problem, design, security analysis, results, limitations.
- Record a backup demo video. BLE demos can fail live.
- Update the ProxiSync slides with the changes (device key pair, rotating token).

Done when: Rehearsed demo, final report and slides.

4. Roadmap (7 weeks)

Week	Focus	Milestone
1	Feasibility spike: BLE broadcast, scan, RSSI measurements, beacon decision	RSSI table and written decision
2	Backend and database running, extra endpoints, Postman collection	Simulator passes, endpoints verified
3	Student app: login, device registration, scan and submit	Student marks attendance on a real phone
4	Student app polish. Teacher app: start session, broadcast token	Teacher starts a session, student marks
5	Teacher live view, close, manual add, reports	Full class flow works
6	Security hardening, per-room calibration, attack tests, edge cases. Feature freeze at the end of the week	All attacks blocked
7	Field test, report, slides, demo video, two rehearsals	Final demo ready

Suggested team split (5 members)

- BLE and mobile lead: advertising, scanning, RSSI sampling.
- Student app developer.
- Teacher app developer.
- Backend and security: API, JWT, signature verification, token logic.
- Database, testing and documentation: schema, test cases, report, demo video.

Risks and fallbacks

Risk	Fallback
Phone BLE advertising unreliable	Switch to an ESP32 beacon in the room
RSSI too noisy	More samples, looser threshold, optional WiFi BSSID check
iOS restrictions	Limit demo to Android and state it as a limitation
Schedule slips	Drop WiFi check, CSV export and alerts. Keep all security layers
Live demo fails	Play the recorded video and show server logs

5. System architecture and data

5.1 Components

Component	Responsibility
Teacher app	Login, pick course and room, start and close sessions, broadcast the rotating token over BLE, live attendance list, manual add, reports.
Student app	Login, one-time device registration, scan for the session beacon, sample RSSI, sign and submit, view own attendance.
Backend (Node.js, Express)	Auth, session management, token generation, signature verification, all validation checks, reports.
Database (PostgreSQL)	Users, device public keys, courses, enrollments, rooms, sessions, attendance (including rejected attempts for audit).

5.2 Database tables

Table	Key columns
users	id, role (student or teacher), name, roll_no, email, password_hash
devices	user_id, public_key, status (active or revoked). Only one active device per user.
courses	id, code, name, division
enrollments	course_id, student_id
rooms	id, name, rssi_threshold (default -75)
sessions	id, course_id, teacher_id, room_id, secret, starts_at, ends_at, closed
attendance	session_id, student_id, status (present, rejected, manual), rssi_median, reject_reason, note, marked_at. One present or manual row per student per session.

5.3 End-to-end flow of one attendance mark

- Teacher taps Start. The app calls POST /sessions and receives the session id.
- Every 15 seconds the teacher app calls GET /sessions/:id/token and updates the BLE advertisement with the new token.
- Student opens Mark Attendance. The app scans, finds the beacon, reads session id and token, and collects at least 3 RSSI readings over about 3 seconds.
- The app builds the string `session_id|token|ts`, signs it with the private key, and calls POST /attendance.
- The server runs the seven checks and answers ok true, or ok false with a reason code.
- The teacher's live list refreshes every 3 seconds and shows the new student.

6. Backend API contract (for the frontend)

Base URL during development: `http://YOUR_LAPTOP_IP:3000`. All bodies are JSON. After login, send the header `Authorization: Bearer TOKEN` on every protected call. The JWT lasts 12 hours. A 401 response means the token is missing or expired: send the user back to the login screen.

6.1 Endpoints that already exist

Method and path	Who	Request body	Success response
POST /auth/register	Public	role, name, roll_no, email, password	{ id }
POST /auth/login	Public	email, password	{ token }. Role is inside the JWT, so decode it or read it from a /me call (see 6.3).
POST /devices/register	Student	public_key (PEM string)	{ ok: true }. Returns 409 if a device is already bound.
POST /devices/reset/:studentId	Teacher	none	{ ok: true }. Revokes the student's device so they can re-register.
POST /sessions	Teacher	course_id, room_id, duration_minutes	{ id, ends_at }
GET /sessions/:id/token	Teacher	none	{ token, expires_in } (seconds left in this slot). 404 if session not active.
POST /sessions/:id/close	Teacher	none	{ ok: true }
POST /attendance	Student	session_id, token, ts, rssi_samples, signature	{ ok: true } or { ok: false, reason }
GET /sessions/:id/attendance	Teacher	none	Array of { name, roll_no, status, rssi_median, marked_at }
POST /sessions/:id/manual	Teacher	student_id, note (required)	{ ok: true }
GET /reports/course/:id.csv	Teacher	none	CSV text: roll_no, name, attended, total, percent
GET /me/attendance	Student	none	Array of { course, total, attended }. Counts may arrive as strings, so convert with Number().

6.2 Attendance request in detail

```
POST /attendance
{
  "session_id": 12,
  "token": "a3f9c1d27b44",            // 12 hex characters read from BLE
  "ts": 1760352000,                   // current time, UNIX SECONDS (not milliseconds)
  "rssi_samples": [-61, -63, -60, -62],   // at least 3 numbers
  "signature": "MEUCIQ..."            // base64 signature of the string below
}
String that must be signed (exactly, no spaces):
  "<session_id>|<token>|<ts>"   e.g.  "12|a3f9c1d27b44|1760352000"
```

Signature rules: SHA-256 with the device private key, output base64. The backend verifies with the stored public key using Node's `crypto.verify`. The public key must be exported as PEM in SPKI format (begins with `-----BEGIN PUBLIC KEY-----`). Test signing against the backend in Week 1 or 2 with one fixed example, because encoding mismatches (DER versus raw signatures) are the most common failure.

6.3 Endpoints the frontend needs that do NOT exist yet

The backend developer must add these before the apps can show real data. Until then the frontend should use mock data (Section 9.4).

Endpoint	Purpose	Suggested response
GET /me	Return the logged-in user so the app knows the role and name	{ id, role, name, roll_no }
GET /courses	Teacher: courses they teach. Student: enrolled courses	[{ id, code, name, division }]
GET /rooms	Room list for the Start Session screen	[{ id, name }]
GET /courses/:id/students	Teacher: students for the manual-add picker	[{ id, name, roll_no }]
GET /sessions?status=active	Teacher: resume an active session after app restart	[{ id, course_id, ends_at }]
GET /me/device	Student: does an active device exist?	{ registered: true or false }

7. Frontend specification: overview

For the frontend developer. You are building one React Native project that contains both roles. After login the app opens the student navigator or the teacher navigator depending on the user's role. Target: Android first (API 26 and above). The backend contract in Section 6 is fixed, so you can build screens against mock data now and switch to the real API later.

7.1 Recommended libraries

Purpose	Suggested library	Note
Navigation	@react-navigation/native, native-stack, bottom-tabs	Auth stack plus a tab navigator per role
HTTP	axios	One instance with the base URL and an interceptor that adds the token and handles 401
Secure storage of JWT	react-native-keychain or expo-secure-store equivalent	Never use plain AsyncStorage for the token
BLE scan (student)	react-native-ble-plx	Provides RSSI per scan result
BLE advertise (teacher)	An Android BLE advertiser library for React Native	Choose and test in Week 1. If none works reliably, a small native Android module is the fallback.
Device key pair and signing	A library that exposes Android Keystore signing, or a small native module	Private key must never leave the device. Verify the signature against the backend early.
Permissions	react-native-permissions	Bluetooth scan, advertise, connect and location where required
State	React Context or Zustand	Keep it simple: auth state and current session

These are suggestions. The BLE advertising and keystore libraries in particular must be tried in the Week 1 spike, because library support changes. Do not commit to one until it works on a real phone.

7.2 Suggested folder structure

```
proxisync-app/
  src/
    api/            client.js (axios), auth.js, sessions.js, attendance.js
    ble/            scanner.js, advertiser.js, payload.js
    security/       deviceKey.js (create key, export public key, sign)
    navigation/     AuthStack.js, StudentTabs.js, TeacherTabs.js
    screens/
      auth/         LoginScreen.js, RegisterScreen.js
      student/      HomeScreen.js, DeviceSetupScreen.js, MarkAttendanceScreen.js,
                    MyAttendanceScreen.js
      teacher/      HomeScreen.js, StartSessionScreen.js, LiveSessionScreen.js,
                    ManualAddScreen.js, ReportsScreen.js
    components/     Button, Card, StatusBanner, CountdownRing, StudentRow, EmptyState
    context/        AuthContext.js, SessionContext.js
    utils/          errors.js (reason code to message), time.js
    mocks/          mockApi.js
```

7.3 Navigation map

Role	Screens
Not logged in	Login, Register
Student, first time	Device Setup (forced, cannot be skipped)
Student tabs	Home, Mark Attendance, My Attendance

Role	Screens
Teacher tabs	Home, Live Session (active only), Reports
Teacher stack screens	Start Session, Manual Add

8. Screen-by-screen specification

8.1 Shared screens

Login screen

Purpose: Authenticate and route by role.

Elements

- App name and logo at top.
- Email field (keyboard type email) and password field with show/hide toggle.
- Login button, disabled while the request runs, shows a spinner.
- Link: 'New here? Register'.

States and behaviour

- Validation before sending: valid email format, password not empty.
- 401 from server: show 'Wrong email or password' under the form.
- Network failure: show 'Cannot reach server. Check your connection.'
- On success store the JWT securely, fetch /me, then route: teacher goes to teacher tabs. Student with no registered device goes to Device Setup, otherwise to student tabs.

API calls: POST /auth/login, GET /me, GET /me/device

Register screen

Purpose: Create a student or teacher account.

Elements

- Role selector (Student or Teacher).
- Name, email, password, confirm password.
- Roll number field, shown only for Student.

States and behaviour

- Password at least 8 characters and both fields match.
- Duplicate email: server error, show 'This email is already registered'.
- On success go back to Login with a success banner.

API calls: POST /auth/register

8.2 Student screens

Device Setup screen (first login only)

Purpose: Bind this phone to the student account. Without it attendance cannot be marked.

Elements

- Short explanation: 'Your account will be locked to this phone so nobody else can mark attendance for you.'
- Button 'Register this device'.
- Progress states: creating key, sending to server, done.

States and behaviour

- On tap: generate key pair in the secure keystore, export the public key as PEM, send it.
- 409 response: show 'A device is already registered. Ask your teacher to reset it.' with a Logout button.
- Success: show a green confirmation and continue to Home.
- The screen cannot be skipped or backed out of until registration succeeds.

API calls: POST /devices/register

Student Home

Purpose: Quick overview and the main action.

Elements

- Greeting with the student's name.
- Large primary button 'Mark Attendance'.
- Cards with attendance percentage per subject (preview of My Attendance). Percentage below 75 is shown in red.

States and behaviour

- Pull to refresh.
- Empty state if no courses: 'You are not enrolled in any course yet.'

API calls: GET /me/attendance

Mark Attendance screen (most important screen)

Purpose: Scan for the teacher's beacon and submit.

Elements

- Large status area in the middle, a status icon and one line of text.
- Bluetooth and permission status banner at the top.
- Cancel button.

States and behaviour

- **Idle:** 'Ready. Tap Start' with a Start Scan button.
- **Checking:** verify Bluetooth is on and permissions granted. If off, show 'Turn on Bluetooth' with a button that opens settings. If permission denied, explain why it is needed and offer to request it again.
- **Scanning:** animated radar, text 'Looking for your class...'. Timeout after 20 seconds with 'Class not found. Move closer to the teacher and try again' and a Retry button.
- **Found:** show the subject name if available. Collect at least 3 RSSI readings over about 3 seconds ('Verifying you are in the room...').
- **Submitting:** spinner, 'Marking attendance...'. Disable the Cancel button.
- **Success:** big green tick, 'Attendance marked', time, subject. Button 'Done'.
- **Failure:** red icon and the friendly message for the reason code (Section 8.4) with a Try Again button where retrying makes sense.
- If the token has rotated while scanning (TOKEN_EXPIRED_OR_INVALID), automatically rescan once before showing an error.
- Never allow double tap on submit. Set ts at the moment of signing, in seconds.

API calls: POST /attendance (BLE parameters in Section 9)

My Attendance screen

Purpose: Student sees their record per subject.

Elements

- List of subjects, each with attended count, total sessions, a percentage and a progress bar.
- Overall percentage at the top.
- Red highlight and the note 'Below 75%' when under the limit.

States and behaviour

- Pull to refresh. Handle total equal to 0 as 0 percent without dividing by zero.
- Counts arrive as strings: convert before calculating.

API calls: GET /me/attendance

8.3 Teacher screens

Teacher Home

Purpose: Entry point for taking attendance.

Elements

- Greeting and the list of the teacher's courses as cards.
- Each card has a Start Attendance button.
- If an active session exists, a banner 'Session in progress, Resume' at the top.

States and behaviour

- Pull to refresh.
- Empty state if the teacher has no courses.

API calls: GET /courses, GET /sessions?status=active

Start Session screen

Purpose: Configure and start a session.

Elements

- Course (pre-filled), room picker, duration selector (5, 10, 15 minutes).
- Checklist: Bluetooth on, permissions granted.
- Start button.

States and behaviour

- On Start: create the session, then begin advertising. If advertising fails, show 'Could not start Bluetooth broadcast' and close the session so no empty session stays open.
- Success: open Live Session.

API calls: POST /sessions, GET /rooms

Live Session screen

Purpose: Run the class session and watch attendance arrive.

Elements

- Header: course name, room, time remaining until the session ends.
- Countdown ring showing seconds until the broadcast token rotates (from expires_in).
- Counter 'Present: N of M'.
- Scrolling list of present students (name, roll number, time marked, tag 'Manual' where relevant), newest at the top.
- Buttons: 'Add manually' and 'Close session' (confirmation dialog).

States and behaviour

- Every 15 seconds fetch a new token and update the advertisement. If a token fetch fails, retry once, then show a red banner 'Broadcast interrupted'.
- Refresh the list every 3 seconds. Keep the screen awake while the session is active.
- If the session time ends, show 'Session ended' and disable broadcasting.
- On Close: call close, stop advertising, show a summary with the final count and a button to export or go home.

API calls: GET /sessions/:id/token, GET /sessions/:id/attendance, POST /sessions/:id/close

Manual Add screen

Purpose: Mark a student present when technology fails.

Elements

- Search box and list of enrolled students, tap to select.
- Note field (required), for example 'Phone battery dead'.
- Confirm button.

States and behaviour

- The note is mandatory: show an inline error if empty.
- Students already present are greyed out.

API calls: GET /courses/:id/students, POST /sessions/:id/manual

Reports screen

Purpose: Export attendance for a course.

Elements

- Course picker and a 'Download CSV' button.
- Preview table: roll number, name, attended, total, percent.

States and behaviour

- Percentage below 75 shown in red. The CSV is fetched as text and saved or shared using the Android share sheet.

API calls: GET /reports/course/:id.csv

8.4 Error reason codes and what to show the user

Reason code	Message for the student	Retry?
SESSION_INACTIVE	This attendance session is closed.	No
NOT_ENROLLED	You are not enrolled in this course. Contact your teacher.	No
STALE_REQUEST	Your phone's clock looks wrong. Set the time to automatic and try again.	Yes
NO_DEVICE_BOUND	No device is registered. Open Device Setup.	No
BAD_SIGNATURE	This phone is not the one registered for your account.	No
TOKEN_EXPIRED_OR_INVALID	The code expired. Trying again...	Yes (auto once)
TOO_FAR	You seem too far from the classroom. Move closer and try again.	Yes
ALREADY_MARKED	Your attendance is already marked for this session.	No (show as success)
Network error	Cannot reach the server. Check your connection and try again.	Yes

9. BLE, signing and technical details

9.1 BLE advertisement payload (fixed contract between the two apps)

The teacher app broadcasts a fixed ProxiSync service UUID. The student app filters scan results by that UUID, so it ignores all other Bluetooth devices. The payload carries the session id and the current token in service data (10 bytes in total, which fits in a normal BLE advertisement).

Bytes	Content	Encoding
0 to 3	Session id	Unsigned 32-bit integer, big endian
4 to 9	Token	The 12 hex characters of the token converted to 6 raw bytes

The team must agree one service UUID (generate once, for example with an online UUID generator, and keep it in a shared constants file used by both apps). The teacher app must refresh the advertisement every 15 seconds with the newest token.

9.2 Student RSSI sampling

- While the beacon is in range, record each scan result's RSSI. Collect at least 3 samples (aim for about 6 over 3 seconds) and send them all in rssi_samples.
- Discard impossible values (above 0 or below -110).
- Do not average on the phone. Send raw samples. The server takes the median.

9.3 Android permissions

Android version	Needed permissions
Android 12 and newer	BLUETOOTH_SCAN (student), BLUETOOTH_ADVERTISE (teacher), BLUETOOTH_CONNECT
Android 11 and older	BLUETOOTH, BLUETOOTH_ADMIN, ACCESS_FINE_LOCATION (and location services must be switched on for scanning)

Show a plain-language explanation before each permission prompt, for example 'Bluetooth is used only to detect that you are in the classroom.' Handle the 'denied' and 'never ask again' cases by sending the user to the app settings.

9.4 Working before the backend and BLE are ready

- Create `mocks/mockApi.js` that returns the same shapes as Section 6 after a short delay (for example 600 ms). Switch with one flag, USE_MOCKS.
- Include mock failures: let the mock return each reason code so every error state in Section 8.4 can be seen.
- Create a fake BLE scanner module that simulates 'found beacon' after 3 seconds, so the Mark Attendance screen can be built and demonstrated without hardware.
- Replace the mocks with real modules one at a time once Step 1 and Step 2 are complete.

9.5 UI and quality guidelines

- One primary action per screen, large touch targets (at least 48 dp), readable text (at least 14 sp).
- Show a loading state for every network call and never leave the user with a blank screen.
- Use clear colours for status: green success, red failure, amber warning. Do not rely on colour alone, always add an icon or text.
- Keep the screen awake during a teacher session and during scanning.

- Handle app going to background during scanning: stop the scan and return to Idle.
- Do not log tokens, passwords or private key material.

10. Testing checklist

Area	Test	Expected result
Functional	Valid student marks attendance	Success screen, appears on teacher's live list
Functional	Mark twice	Shown as already marked, no duplicate row
Functional	Mark after session closed	SESSION_INACTIVE message
Attack	Token sent by chat to a student outside the room	TOO_FAR, or token expires within about 40 seconds
Attack	Login on a friend's phone	BAD_SIGNATURE
Attack	Resend an old captured request	STALE_REQUEST
Attack	Student standing in the corridor	TOO_FAR
Edge	Bluetooth switched off	Clear prompt to turn it on
Edge	Permission denied	Explanation and a link to settings
Edge	No internet during submit	Network error message, retry works inside the time window
Edge	Teacher phone battery dies	Manual add works, report still correct
Accuracy	Field test with 15-20 students	Record false accepts, false rejects, average time to mark

Definition of done for the minor project

- Student and teacher apps run on real Android phones against the real server.
- All four attack tests are blocked with the right reason code.
- Field test numbers collected and included in the report.
- Backup demo video recorded.
- Report and slides finished.

After the minor project (major project path)

- Admin dashboard (web) and cloud deployment.
- Moodle or ERP integration for syncing attendance.
- iOS support and multi-room beacon management.
- Analytics: low-attendance trends and defaulter lists.